

Infotact Technical Internship Program: Advanced Data Science & Machine Learning Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, algorithmic directives, and sprint-based modeling roadmaps for three advanced Data Science and Machine Learning (DS/ML) projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for AI and ML interns entering the Infotact startup ecosystem.

In 2026, industry expectations for Data Scientists have evolved beyond basic tabular regression. Modern AI teams expect proficiency in end-to-end data pipelines, complex spatiotemporal modeling, reinforcement learning, and advanced feature engineering. The following projects focus on highly sought-after industry solutions: IoT Predictive Maintenance, Reinforcement Learning for Dynamic Pricing, and Geospatial Real Estate Valuation.

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. **The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.**

Project 1: Manufacturing & Automotive - Contextual Predictive Maintenance (IoT Edge AI)

Executive Problem Statement

Predictive maintenance offers the potential to drastically reduce unexpected breakdowns and lower fleet or factory maintenance costs. However, most existing ML systems rely exclusively on internal diagnostic signals and fail in real-world deployment. A machine's failure is rarely isolated; it is heavily influenced by external factors such as weather conditions, factory load, or traffic density.

The objective of this project is to build an advanced contextual data fusion framework. The intern will integrate internal IoT telemetry (sensor data like vibration, current, and temperature) with external environmental signals acquired via third-party APIs or Vehicle-to-Everything (V2X) logs. The goal is to accurately predict mechanical failures before they happen using robust ensemble modeling.

Business Objectives and Key Performance Indicators

The strategic vision is to transition industrial maintenance from a reactive "break-fix" model to a highly accurate proactive model.

Success is measured by the model's robustness against noisy industrial data. Because machine failures are rare, the dataset will be highly imbalanced. The intern must optimize the Macro

F_1 score, ensuring the model maintains high predictive accuracy (Macro $F_1 \geq 0.85$) even when the sensor data experiences moderate noise interference.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Fleet/Plant Manager	Minimizing unexpected downtime and optimizing repair schedules.	Reviews the predictive dashboard to identify which specific machines or vehicles have a $>80\%$ probability of failing in the next 7 days.
Reliability Engineer	Understanding the root cause of predicted failures.	Analyzes the feature importance charts (e.g., SHAP values) to see if external temperature or internal vibration is driving the anomaly.

Minimum Viable Product Specifications

The foundational feature is the Contextual Data Fusion Pipeline. The intern must merge high-frequency IoT sensor time-series data with external API data (like weather or load conditions) using complex Pandas windowing operations.

The core deliverable is the Classification Pipeline. The intern will utilize advanced gradient boosting algorithms (like LightGBM) combined with strict Synthetic Minority Over-sampling Technique (SMOTE) applied *only* within training folds during cross-validation to prevent data leakage.

Four-Week Engineering Roadmap

Week 1: IoT Telemetry Ingestion and Signal Processing

The intern will ingest a complex industrial dataset (e.g., AI4I Predictive Maintenance Dataset). Using Python, the intern must handle massive time-series logs, calculating rolling means, standard deviations, and signal variances over specific operational windows to create baseline features.

Week 2: Contextual Data Fusion and Feature Engineering

Week two focuses on data enrichment. The intern must simulate or integrate external context (e.g., ambient temperature, load density) and merge it with the internal sensor telemetry based on precise timestamps. The intern will conduct an ablation study to prove mathematically that adding external features improves predictive power.

Week 3: Imbalanced Classification and LightGBM Modeling

The third week introduces advanced ML modeling. The intern will set up a 5-fold stratified cross-validation pipeline. To handle the fact that actual machine failures account for $<2\%$ of

the data, the intern must implement SMOTE carefully within the cross-validation loop. They will train a LightGBM classifier to predict impending faults.

Week 4: Noise Sensitivity Analysis and Threshold Tuning

The final week mimics real-world deployment challenges. The intern will inject synthetic noise into the test dataset to empirically characterize model robustness. They will plot Precision-Recall curves and adjust the decision threshold to balance false alarms vs. missed maintenance windows. The entire pipeline must be documented on GitHub.

Project 2: Travel & Hospitality - Reinforcement Learning for Dynamic Pricing

Executive Problem Statement

Selling finite inventory over a limited time period—such as airline tickets, hotel rooms, or concert seats—is a complex mathematical challenge. Static pricing or simple rule-based algorithms fail to account for fluctuating market demand, competitor pricing, and the shrinking time left in the selling season. This results in either unsold inventory (spoilage) or selling out too early at suboptimal prices.

The objective of this project is to move beyond traditional supervised learning and build an autonomous pricing agent using Reinforcement Learning (RL). The intern will train an AI agent that learns optimal pricing policies through continuous interaction with a simulated market environment.

Business Objectives and Key Performance Indicators

The strategic goal is continuous Revenue Maximization. The RL agent must learn to balance two competing forces: keeping prices high to maximize margin, and dropping prices to ensure the fixed inventory clears before the "flight departs" or the "hotel date passes."

Success will be quantified by the total episodic reward (Total Revenue generated per season). The RL agent must mathematically outperform traditional baseline strategies (like Time-Based Pricing or flat Demand-Based Pricing).

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Revenue Manager	Maximizing Yield per Available Room (RevPAR) or Seat.	Monitors the RL agent's pricing adjustments on a dashboard and sets upper/lower safety bounds for the algorithm.

Data Scientist	Stable and convergent reinforcement learning models.	Tunes reward functions and state-space representations to ensure the RL policy does not exploit edge cases detrimentally.
-----------------------	--	---

Minimum Viable Product Specifications

The foundational feature is the Market Simulation Environment. The intern must build a custom OpenAI Gym (or Gymnasium) environment representing the airline/hotel booking process, where the "State" consists of `[remaining_inventory, days_until_departure]`, and the "Action" is the `price_level`.

The core deliverable is the RL Agent. The intern will implement tabular Q-Learning or a Deep Q-Network (DQN) to discover the optimal pricing policy, explicitly modeling the long-term objective of maximizing cumulative revenue.

Four-Week Engineering Roadmap

Week 1: Designing the MDP and Gym Environment

The intern will formulate the pricing problem as a Markov Decision Process (MDP). They will code a custom Python environment subclassing `gym.Env`. The intern must design a stochastic demand function where customer purchase probability decreases as the agent increases the price, but also fluctuates based on the time left.

Week 2: Baseline Strategies and Q-Learning

Week two serves as the algorithmic baseline. The intern will first code naive heuristic agents (e.g., "sell at fixed price", "discount by \$10\%\$ every day"). Then, they will implement a fundamental Q-Learning algorithm, creating a Q-table that maps every possible (inventory, time) state to the most profitable price action.

Week 3: Deep Reinforcement Learning (DQN/PPO)

The third week introduces deep learning. Because real-world state spaces are massive (continuous time, complex inventory), the intern will replace the Q-table with a Neural Network (Deep Q-Network). The intern must handle the exploration-exploitation trade-off using an epsilon-greedy strategy and implement experience replay to stabilize training.

Week 4: Policy Evaluation and Business Dashboard

The final sprint involves translating RL metrics into business value. The intern will run 1,000 simulated booking seasons to evaluate the DQN agent against the naive baselines. They will use Matplotlib/Seaborn to plot the "Price Trajectory" over time, proving that the agent learned complex behaviors (like dropping prices near the deadline to clear remaining stock). The custom Gym environment and trained weights must be pushed to GitHub.

Project 3: Construction & Real Estate - Geospatial Valuation via Spatial Embeddings

Executive Problem Statement

Traditional Automated Valuation Models (AVMs) used in real estate heavily rely on tabular features (number of bedrooms, square footage) but fail to capture the complex spatial relationships of a housing market. A house's value is deeply influenced by its neighbors, nearby

amenities, and localized socio-economic factors. Simple linear regression or tree-based models cannot natively process these geo-spatial dependencies.

This project requires the intern to build a state-of-the-art Real Estate Valuation engine. The intern will move beyond standard Machine Learning by utilizing Spatial Embeddings, K-Nearest Neighbor (KNN) graphs, or Graph Neural Networks (GNNs) to mathematically model how surrounding properties affect a target house's price.

Business Objectives and Key Performance Indicators

The primary objective is hyper-accurate property appraisal for real estate firms or automated lending platforms.

Success is measured by minimizing the Mean Absolute Percentage Error (MAPE) against holdout datasets. The intern must prove that injecting spatial dependencies (graph networks or spatial embeddings) outperforms traditional machine learning baselines like XGBoost or pure Linear Regression.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Real Estate Appraiser	Data-driven, unbiased property valuations based on comparable local sales.	Inputs a property address into the system; the model outputs a predicted price along with the top 5 most influential neighboring properties.
Investment Strategist	Identifying undervalued neighborhoods or "hotspots."	Reviews spatial heatmaps generated by the embeddings to guide large-scale commercial acquisitions.

Minimum Viable Product Specifications

The application foundation is the Geospatial Data Wrangling module. The intern must use tools like **GeoPandas** and **Shapely** to process latitude/longitude coordinates, calculate Haversine distances, and construct neighborhood graphs connecting nearby properties.

The core deliverable is the Attention-Based or Graph-Based Valuation Model. The intern will build a deep learning architecture (using PyTorch or DGL) that aggregates the pricing data of a property's immediate neighbors, utilizing an attention mechanism to weigh certain neighbors more heavily than others.

Four-Week Engineering Roadmap

Week 1: Geospatial Data Acquisition and Processing

The intern must acquire a real estate dataset containing exact lat/long coordinates (e.g., King County Housing Data). Using Python and GeoPandas, they will clean the data, normalize

extreme outliers in house prices, and plot the data geographically on interactive maps (using [Folium](#) or [Kepler.gl](#)) to visually inspect spatial pricing trends.

Week 2: Feature Engineering and Baseline ML

Week two establishes the benchmark. The intern will engineer standard tabular features (age of the house, distance to city center) and train an XGBoost regressor as the baseline model. The intern will calculate the baseline MAPE and RMSE, documenting the model's limitations when predicting prices in rapidly gentrifying neighborhoods.

Week 3: Spatial Embeddings and Graph Construction

The third week introduces spatial mathematics. The intern will convert the dataset into a Graph structure, where each house is a "Node" and edges connect the house to its $\$K\$$ -nearest neighbors based on physical distance. The intern will generate spatial embeddings to represent the localized neighborhood context mathematically.

Week 4: GNN / Attention Modeling and Geospatial Dashboard

The final sprint involves advanced Deep Learning. The intern will train a Graph Neural Network (GNN) or an Attention-Based Spatial model on the constructed graph. They will compare the new MAPE against the XGBoost baseline to prove the efficacy of spatial dependencies. Finally, the intern will deploy a Streamlit app visualizing the predicted price disparities on a map. Code must be rigorously version-controlled on GitHub.

Universal Data Science and GitHub Version Control Guidelines

To simulate a rigorous, enterprise-grade AI research environment indicative of top-tier deep tech companies in Bengaluru, strict adherence to modern ML engineering practices and version control is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution. Machine Learning is not merely the final trained model; it is the iterative, documented process of experimentation.

Mandatory Evaluation Protocol

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading a single massive Jupyter Notebook containing data cleaning, GNN training, and evaluation in one push on the final day—will result in immediate disqualification.

Instructions for Tracking Week-Wise Production with Timestamps

Data Science and Machine Learning require a specific, rigorous approach to version control. To pass the evaluation, interns must utilize GitHub's native tools to explicitly track their weekly production.

1. Setup GitHub Projects (Kanban Board)

At the start of Week 1, the intern must navigate to their repository's **Projects** tab and create a new Kanban board (To Do, In Progress, Done).

- **Mandatory Action:** The intern must break down the 4-week roadmap into individual GitHub Issues (e.g., Issue #1: "Code Haversine distance function", Issue #2: "Train DQN Agent for dynamic pricing").

- **Timestamping:** GitHub automatically timestamps when an issue is created, moved, and closed. Managers will audit this board to verify a steady, weekly cadence of ML experimentation.

2. Commit Frequency and Semantic Issue Referencing

Commits act as the immutable, timestamped ledger of an ML Engineer's thought process. Interns are expected to commit their preprocessing scripts, custom Gym environments, and notebook updates frequently (3-5 times per active development day).

- **Mandatory Action:** Every commit message must reference the specific GitHub Issue it solves using semantic messaging.
- **Format Example:** `feat: implement SMOTE inside cross-validation folds (fixes #4)` or `model: tune PPO hyperparameters for pricing agent (fixes #6)`
- By appending `(fixes ##)`, GitHub automatically links the timestamped code commit to the project management board, providing undeniable proof of week-over-week contribution.

3. Managing Jupyter Notebooks and Model Weights in Git

Jupyter Notebooks (`.ipynb`) and trained model weights (`.pt`, `.h5`) are notoriously difficult to track in Git.

- **Mandatory Action (Notebooks):** Before committing a Jupyter Notebook, the intern **must clear all cell outputs** (Kernel -> Restart & Clear Output) or use a pre-commit hook like `nbstripout`. This ensures the Git history tracks the actual `code` changes, not massive blocks of serialized JSON data or matplotlib images.
- **Mandatory Action (Weights/Data):** Pushing raw `.csv` files or trained model weight files (which often exceed 100MB) to GitHub is strictly forbidden. The intern must utilize a `.gitignore` file from Day 1 to exclude all `data/` and `models/` directories. Instead, weights should be tracked via external tools (like DVC) or scripts must be provided to re-download the data and re-train the models from scratch.

By strictly enforcing these GitHub tracking specifications, the program ensures that deliverables transcend mere academic exercises, transforming into robust, scalable, and highly professional Data Science portfolios indicative of elite industry standards.